

Phân tích luồng hệ thống RAG Chatbot Thủ tục nhập học HUS 2025

Một số vấn đề chọn lọc về Trí tuệ nhân tạo

Khoa Toán – Cơ – Tin học
Trường Đại học Khoa học Tự nhiên, VNU

Hà Nội, tháng 4 năm 2026

Giới thiệu thành viên

Họ và Tên	Mã Sinh Viên
Nguyễn Xuân Khánh	22001601
Hoàng Anh Minh	22001616

Mục lục

- ➊ Mục tiêu cốt lõi
- ➋ Khó Khăn Và Giải Pháp Thực Tế
- ➌ Tổng quan kiến trúc
- ➍ Đường ống xử lý dữ liệu chuẩn bị (Offline Pipeline)
- ➎ Đường ống vận hành (Online Pipeline)
- ➏ Tổng kết

Mục tiêu cốt lõi

Mục tiêu & Triết lý thiết kế hệ thống

- **Nắm quyền kiểm soát ngữ cảnh:** Tự quản lý lịch sử hội thoại tại máy chủ. API LLM chỉ đóng vai trò bộ não trung tâm.
- **Tri thức phân tầng (Hierarchical Knowledge):** Phân rã dữ liệu văn bản (Chunking) dựa trên cấu trúc ngữ nghĩa thực tế (Phần, Bước), thay vì phân mảnh máy móc theo số ký tự đoạn văn, giúp bảo toàn chặt chẽ logic chuỗi nghiệp vụ.
- **Ưu tiên luật cứng trong một số trường hợp:** Áp dụng nguyên lý ràng buộc hàm (Rule-based) để xử lý các câu hỏi đòi hỏi dữ kiện cứng cáp.
- **Tối ưu truy vấn:** Điều hướng siêu dữ liệu bằng metadata để lọc và khoanh vùng độ khó trước khi phải động tới việc gọi tốn kém LLM ngoài.

Khó Khăn Và Giải Pháp Thực Tế

Khó khăn 1: Hướng triển khai thực tế

- **Vấn đề:** Các API LLM thông thường tối ưu cho trải nghiệm đơn lẻ bằng cách nhồi toàn bộ lịch sử vào Prompt.
- **Giải pháp:**
 - Tự xây dựng bộ quản lý Session và Context riêng biệt tại Backend.
 - Coi LLM là bộ não xử lý thuần túy. Backend sẽ gửi thông tin đưa vào API cho từng luồng chat để trả lời.

Khó khăn 2: Sự xung đột giữa AI và Logic nghiệp vụ

- **Vấn đề:** RAG phụ thuộc vào giá trị ngữ nghĩa của query đầu vào.
- **Giải pháp:**
 - **Rule-based:** Dùng thuật toán định danh (Regex, Intent Matching) để truy vấn chính xác các dữ liệu "cứng".
 - **RAG:** Chỉ dùng để xử lý các câu hỏi mở, giàu sắc thái ngôn ngữ.
 - **Cơ chế điều phối:** Căn cứ vào Query và History để chạy Luật cứng hay RAG.

Khó khăn 3: Trả lời theo quy trình

- **Vấn đề:** Ngôn ngữ thủ tục rất dễ gây nhầm lẫn. Ví dụ: "Bước 2" có thể là bước lớn trong quy trình tổng thể (*Global*), nhưng cũng có thể là một thao tác nhỏ trong cụm hồ sơ (*Local*).
- **Hệ quả:** Nếu máy tính hiểu sai cấp độ (Level), câu trả lời sẽ đi chệch hướng hoàn toàn.
- **Giải pháp (Contextual Metadata Lock):**
 - Định nghĩa Metadata phân tầng cho tài liệu.
 - Trước khi truy vấn, hệ thống thực hiện "khóa mục tiêu" (*Metadata Filtering*) dựa trên phân tích ý định để giới hạn phạm vi tìm kiếm trong đúng phân đoạn văn bản liên quan.

Khó khăn 4: Câu hỏi phụ thuộc ngữ cảnh (Ellipsis)

- **Vấn đề:** Người dùng thường hỏi những câu ngắn, thiếu thông tin như: "Vậy bước tiếp theo làm thế nào?". Nếu tách riêng, câu này có hàm lượng thông tin (Entropy) bằng không, khiến RAG không thể tìm kiếm.
- **Giải pháp (Query Refinement):**
 - Sử dụng lịch sử chat gần nhất để tái cấu trúc lại câu hỏi (Query Rewrite).
 - Biến câu hỏi mơ hồ thành câu hỏi đầy đủ thông tin: "Bước tiếp theo của [Thủ tục nộp học phí] là gì?" trước khi đưa vào bộ máy truy vấn.

Tổng quan kiến trúc

Kiến trúc hệ thống

- **Tầng Giao diện (Frontend - Tương tác người dùng):**
 - Ứng dụng nền Web (HTML/JS).
- **Tầng Máy chủ (Backend - Xử lý tính toán Flask):**
 - Quản lý phiên hội thoại biệt lập (`session_id`) riêng của mọi sinh viên.
 - Trạm luân chuyển (Pipeline) nghiệp vụ RAG: Phân tích → Truy xuất → Sinh văn bản.
- **Tầng Dữ liệu & Bộ nhớ đệm Storage:**
 - **Cơ sở dữ liệu Vector:** Lưu trữ embedding và văn bản, tối ưu lọc qua metadata.
 - **Bộ nhớ cho phiên:** Lưu trữ history.

Luồng xử lý một truy vấn

- ➊ **Tiếp nhận truy vấn:** Client gửi câu hỏi kèm định danh phiên (Session ID) lên hệ thống máy chủ.
- ➋ **Phân tích & Phục hồi ngữ cảnh:** Kết hợp câu hỏi mới và lịch sử hội thoại trước đó nhằm phân tích ý định cốt lõi (Intent) và trích xuất metadata.
- ➌ **Truy xuất RAG + luật cứng (đôi khi):** Áp dụng metadata vừa thu được để thiết lập bộ lọc (Filter), sau đó truy vấn vào Vector DB để trích xuất ra các đoạn văn bản (chunks) có độ tương đồng cao nhất.
- ➍ **Tổng hợp Context & Tạo Prompt:** Chắt lọc các đoạn tài liệu RAG, hợp nhất thành Context nền tảng và gọt giũa vào Prompt hệ thống.
- ➎ **Sinh phản hồi & Trả kết quả:** Đưa Prompt chứa thông tin đã ẩn định vào mô hình LLM để sinh câu trả lời tự nhiên. Đóng gói kết quả, lưu vết lịch sử và trả về Client.

Đường ống xử lý dữ liệu chuẩn bị (Offline Pipeline)

Bước 1: Quét nạp và Phân định tài liệu đầu vào

- Thu nhận tài liệu hỗn hợp trong hệ thống lưu trữ với định dạng TXT, DOCX, PDF.
- Sử dụng bộ trích xuất dữ liệu chuyên biệt (TextExtractor):
 - Bản TXT: Chuyển thể trực tiếp nội dung nguyên trạng.
 - Bản DOCX: Rút nội dung thô qua gói xử lý python-docx.
 - Bản PDF: Đọc nhận diện trực tiếp qua engine PyMuPDF.
- **Khả năng Nhận diện thông minh (Heuristics):** Phân tách nội dung cốt lõi của "Thủ tục hồ sơ khóa mới" dựa vào tín hiệu từ khóa cứng ("PHẦN 1..4", "B1..B4", "Mã trúng tuyển").
- Phủ định (Reject) các tài liệu ngoài luồng, chuyển phân tách sang chế độ xử lý văn bản quy ước chung.

Bước 2: Phân rã dữ liệu (Chunking) theo cấu trúc ngữ nghĩa

Cấp bậc	Quy mô phạm vi	Ví dụ
L0	Toàn cục văn bản	Bản tóm lược toàn cảnh khâu nhập học
L1	Khối thông tin lớn	Bao gồm cả PHẦN 1, PHẦN 2, ...
L2	Nhóm chức năng nhỏ	Khu vực bảng học phí, Trình tự bước nộp
L3	Thông tin vi sinh	Đi sâu tại cụm Bước 1, Ngày 28/8, Tiền...

- **Cấu trúc Phân tầng là cần thiết (Hierarchical):** Ngăn chặn rủi ro tự động bị chặt cut ý như phương thức thông thường

Bước 3: Neo giữ thẻ Siêu dữ liệu (Metadata rẽ nhánh)

- Sau tiến trình cắt, mỗi tiểu khu dữ liệu (Chunk) nhận thẻ định danh mở rộng:
 - Mục tiêu câu hỏi (intent_key, subtype).
 - Trỏ khóa tọa độ (section_id, step_number).
 - Tham số hóa (date, major_norm, time_slot, year, source).
- **Ứng dụng then chốt của Metadata:** Thắt chặt khoanh vùng của các phép tính toán tìm kiếm tốn kém Cosine Similarity, định hướng phân luồng chuẩn xác liệu câu hỏi mang ý nghĩa phạm vi cục bộ hay diện rộng (Local/Global scope resolution).

Bước 4: Nhúng Vector toán học & Tích hợp Database

- Dữ liệu văn bản được đưa qua mô hình embedding để chuyển thành các vector số.
- Các vector sau đó được lưu vào kho dữ liệu `vector_db.json` để phục vụ truy xuất.
- Việc tìm kiếm được thực hiện bằng **độ tương đồng Cosine**:

$$\text{sim}(q, e_i) = \frac{q \cdot e_i}{\|q\| \|e_i\|}$$

- Hệ thống có thể lọc trước theo Metadata (ví dụ: chủ đề, loại tài liệu, nguồn dữ liệu) rồi mới thực hiện tìm kiếm Top-K.
- Cách làm này giúp giảm dữ liệu không liên quan và tăng độ chính xác khi truy xuất.
- Khi dữ liệu được cập nhật hoặc tái lập chỉ mục, bộ nhớ đệm (cache) cần được làm mới để đảm bảo kết quả luôn đúng.

Đường ống vận hành (Online Pipeline)

Bước 5: Quản lý phiên hội thoại

- Nếu chỉ gửi toàn bộ lịch sử chat cho API của LLM, hệ thống có thể gặp 2 vấn đề lớn:
 - **Tồn token không cần thiết:** nhiều đoạn hội thoại cũ hoặc lan man vẫn bị gửi lại.
 - **Dễ giữ nhầm ngữ cảnh:** khi người dùng đổi chủ đề, mô hình có thể vẫn bám vào nội dung trước đó và trả lời sai.
- **Ví dụ:**
 - Câu đầu hỏi về khoa Toán.
 - Câu sau chuyển sang khoa Hóa.
 - Nếu không kiểm soát ngữ cảnh, AI có thể vẫn suy luận theo chủ đề khoa Toán và làm sai thông tin.
- **Cách xử lý ở Backend:**
 - Backend tự quản lý mỗi cuộc hội thoại bằng `session_id`.
 - API chat được xem như một hàm xử lý độc lập, không tự ghi nhớ toàn bộ lịch sử.
 - Chỉ gửi lại khoảng 3–5 lượt hội thoại gần nhất và còn liên quan trực tiếp.

Bước 6 & 7: Nhận diện Ý định & Trích xuất Thực thể

- Khối lượng Analyzer đảm nhiệm vai trò:
 - "Tút tát" và Diễn đạt rõ ràng lại (Query Rewrite) câu hỏi hệt chủ ngữ từ đầu vào người dùng.
 - **Rút trích các khung thực thể:** Tìm kiếm thông qua Regex/Tự điển (ngày tháng, tên khối ngành, "bước máy", "phần thứ bao nhiêu").
- Thiết lập lên **Khung truy vấn gốc (Query frame)** chia rõ:
 - Tuyến hỏi toàn cảnh lộ trình (Global axis): "Kể tên các bước làm thủ tục nhập học?". (Bao quát Phần).
 - Tuyến hỏi cục bộ thao tác (Local axis): "Kể tên các loại giấy tờ ở trong bước nộp hồ sơ thứ 2?". (Khoanh cụm).
- Thuật toán rẽ nhánh theo dõi vết câu hỏi trước và câu hỏi "tiếp theo", suy diễn nội dung bị tỉnh lược chủ ngữ rất sắc bén.

Bước 8: Thuật toán dò tìm đa luồng & Khóa chốt chặn

- Truy xuất văn bản nội bộ được chia ra hai tiến trình đa luồng:
 - **Kiểm tìm khát khe (Filtered target):** Truy gốc theo chính xác nhãn mác cấu trúc cây.
 - **Kiểm tìm lỏng (Broad Net Net):** Dò tìm khái quát thông qua số lượng vector điểm phủ rộng để chắc chắn không bỏ sót trường hợp ngoại lệ.
- Chấm **Điểm chất lượng tham chiếu**. Mô hình so tài cho điểm hai nhanh trên với nhau trước khi xuất kết quả tổng.
- **Bổ sung và nén văn bản vệ tinh (Context Enrichment & Deduplication):** Ghép nhóm chuỗi nội dung cha với nội dung đoạn anh em lân cận, xóa đi các đoạn nhỏ phân mảnh lặp lại và khóa chốt.

Bước 9: Khởi tạo phản hồi (Nói Không phớt lờ hết cho AI)

- **Sụt giảm Uy Tín Thương Hiệu:** Mọi sai lầm tài chính hoặc thời gian từ một thông báo trường ban ra không thể đem đổi lấy cảm giác "giống con người trò chuyện".
- **Triển khai Định tuyến cứng (Deterministic-first Engine):**
 - Nếu là câu hỏi tiền nong truy ra thẳng trường học phí khoa đó: Code python lấy con số JSON nội bộ điền thẳng vô khung chuẩn và trả lời khách hàng. 0 Tồn API phụ thêm AI.
- **Ràng buộc pháp lý bằng Khung JSON (LLM parser as Tool):**
 - Khi sự phức tạp luồng trả lời nằm ngoài khả năng rule lập trình, thì Gọi LLM bằng API thuần. Nhưng ÉP LLM xuất nội dung theo dạng mã máy (JSON String). {
"Dap_{AN}" : "..." , "Tieu_Chuan_{MA}nguon" : "..."}.Vip_hmformat→
Fix.

Bước 10: Vòng Hậu kiểm & Tối ưu đường hiển thị (Validator)

- Lớp màng lọc cuối cùng làm việc rà soát kép:
 - Sàn lọc số ngày tháng/tiết học tiền bạc nếu phát sinh phải KHỚP 100% các kí tự trong Kho Nguồn Truy Cập ban đầu.
 - Mất liên kết, đứt gãy citation → Thuật toán Rollback, trả lời sao lưu mặc định bằng ngôn ngữ máy thô ráp (Tối ưu độ an toàn thay vì cố tỏ ra thông minh sinh ra lỗi ảo giác hệ trọng).
- **Cơ chế tái tạo kiểu phản hồi giọng điệu mềm mại (Style Variation Mode):** Thay khung văn để cho gần gũi, chia xuống chấm điểm (bullet points).
- Nén chốt cache bằng Key dài:
`user::session::intent::query::style`. Trả nguồn Frontend sẵn sàng.

Tổng kết

Lời kết cuối

- Bài trình bày mô hình hoá triệt để và khắc họa kiến trúc phát triển và đưa vào luồng vận hành của một ứng dụng mô hình LLM trong bài toán chuyên môn hóa thủ tục nhập học thực tiễn.
- Khác biệt cốt lõi đến từ: Cấu trúc hóa Chunk văn bản tri thức, Dự đoán Intent, Quản lý ngữ cảnh hạn chế sự thả nổi của LLM.
- Tạo đà để sẵn sàng nghiên cứu sâu hơn về các bài toán triển khai ChatBot ứng dụng.

Cảm ơn sự theo dõi của thầy cô!